

Lab 01

OPNsense Firewall Installation & Initial Configuration

Step-by-Step Lab Report with Screenshots

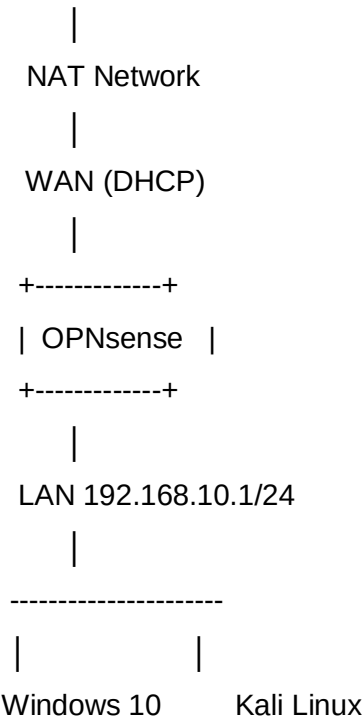
Duration: 3 Hours

Objective: Install OPNsense, configure basic networking, and verify connectivity.

Lab Topology

This lab simulates a small network protected by an OPNsense firewall, with the WAN side representing the internet (via VirtualBox NAT) and the LAN side representing a private internal network (via a VirtualBox Internal Network named “LAN”), shared with two client virtual machines.

Internet



Step 1 – Download OPNsense (10 Minutes)

The OPNsense Community Edition ISO (AMD64, DVD image) was downloaded from the official site (opnsense.org/download) and saved locally for use as the installation media in VirtualBox.

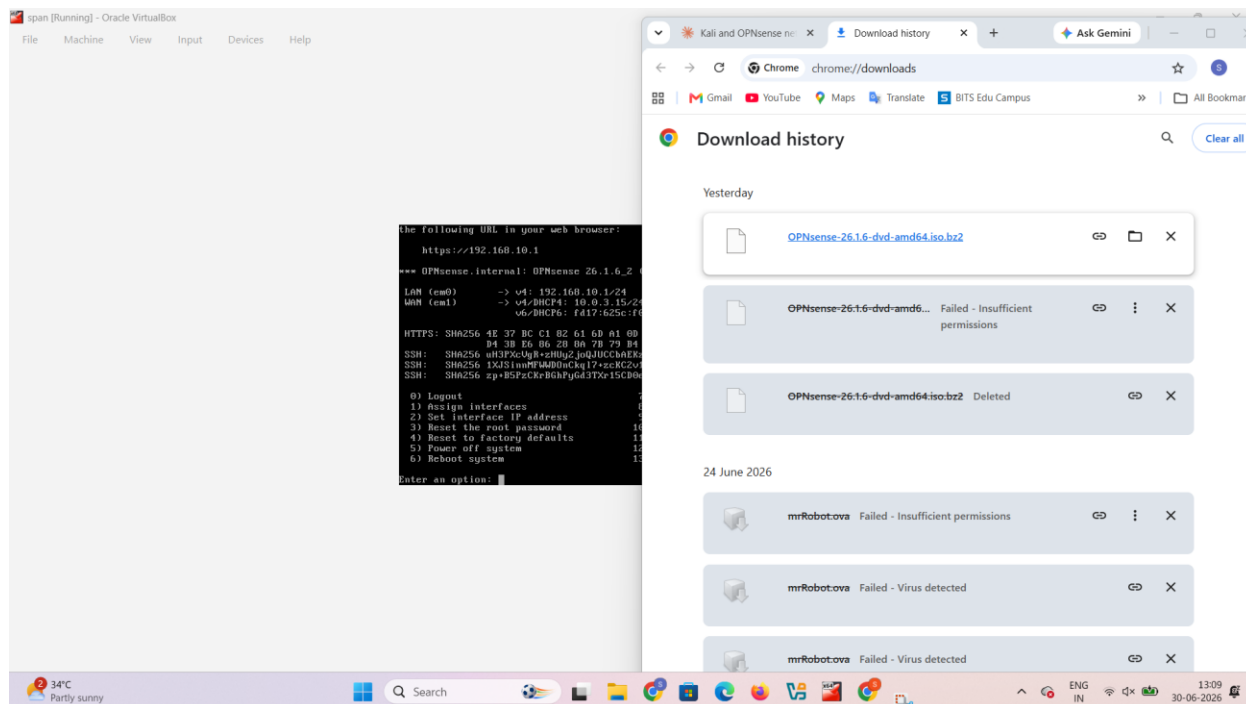


Figure 1.1 – Downloaded OPNsense ISO file (OPNsense-26.1.6-dvd-amd64.iso.bz2) shown in Chrome's download history.

Step 2 – Create a Virtual Machine (20 Minutes)

A new virtual machine named “span” (representing the OPNsense firewall) was created in Oracle VirtualBox with a 20 GB dynamically allocated VDI disk, and the downloaded ISO was attached to the optical drive for installation. A second VM, “sneh” (Kali Linux), was created to act as the client machine on the LAN.

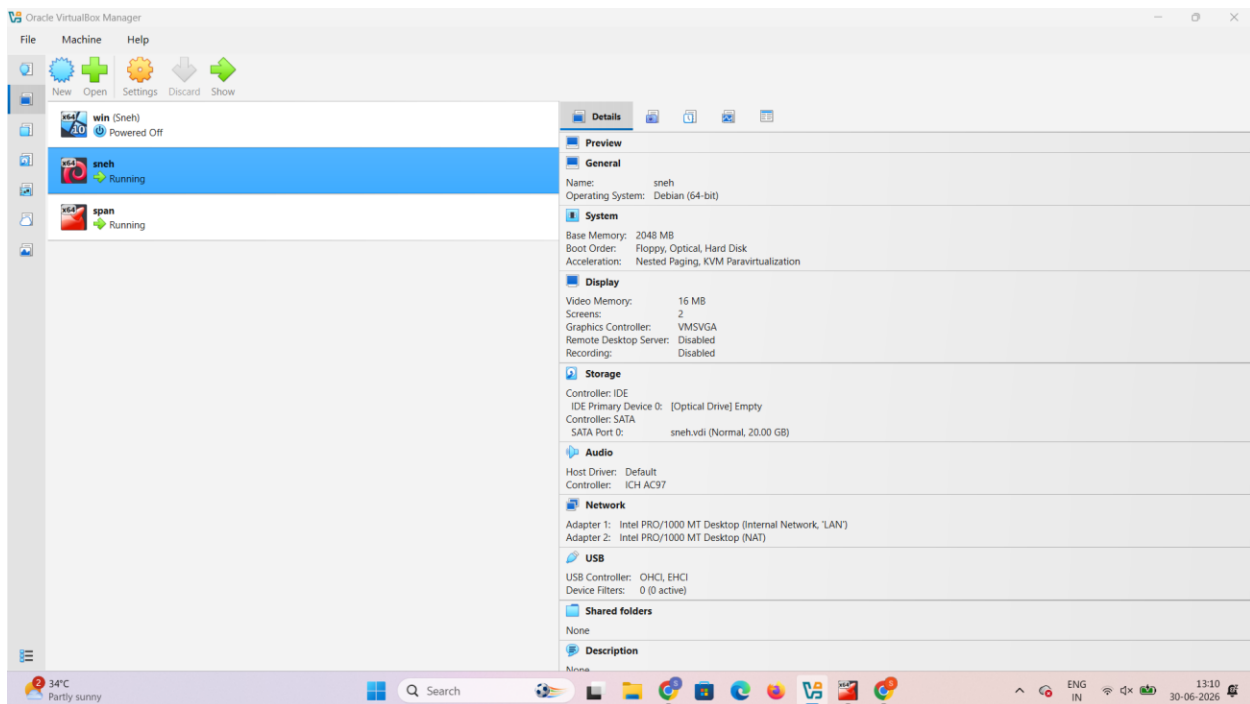


Figure 2.1 – VirtualBox VM list and the sneh (Kali Linux client) VM details: memory, storage, and network adapters.

Step 3 – Configure Network Adapters (15 Minutes)

Each VM was configured with two network adapters so the firewall can sit between the simulated internet and the internal LAN, and so the client VM can reach the LAN.

- **OPNsense (span) – Adapter 1 (WAN):** Attached to NAT, providing outbound internet access.

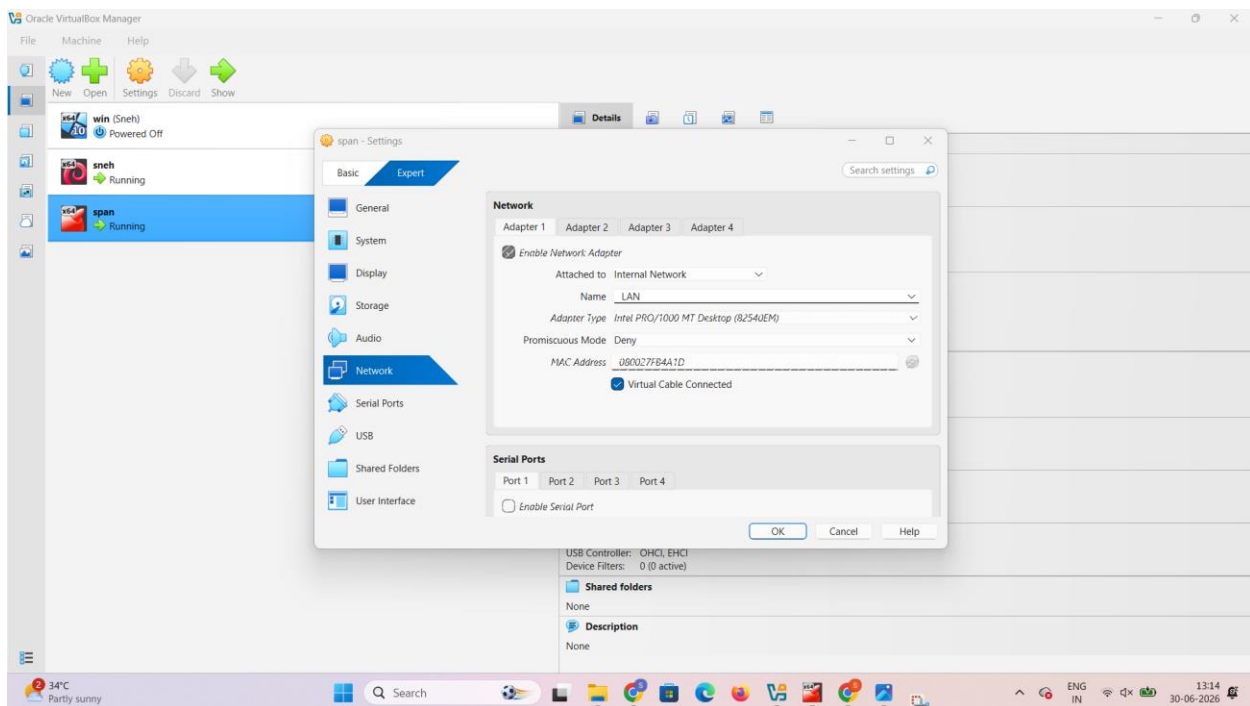


Figure 3.1 – span (OPNsense) Adapter 1 attached to Internal Network “LAN” (used for em0/LAN).

- **Kali Linux (sneh) – Adapter 1 (LAN):** Attached to Internal Network, named “LAN”, so it shares the same internal segment as OPNsense's LAN interface.

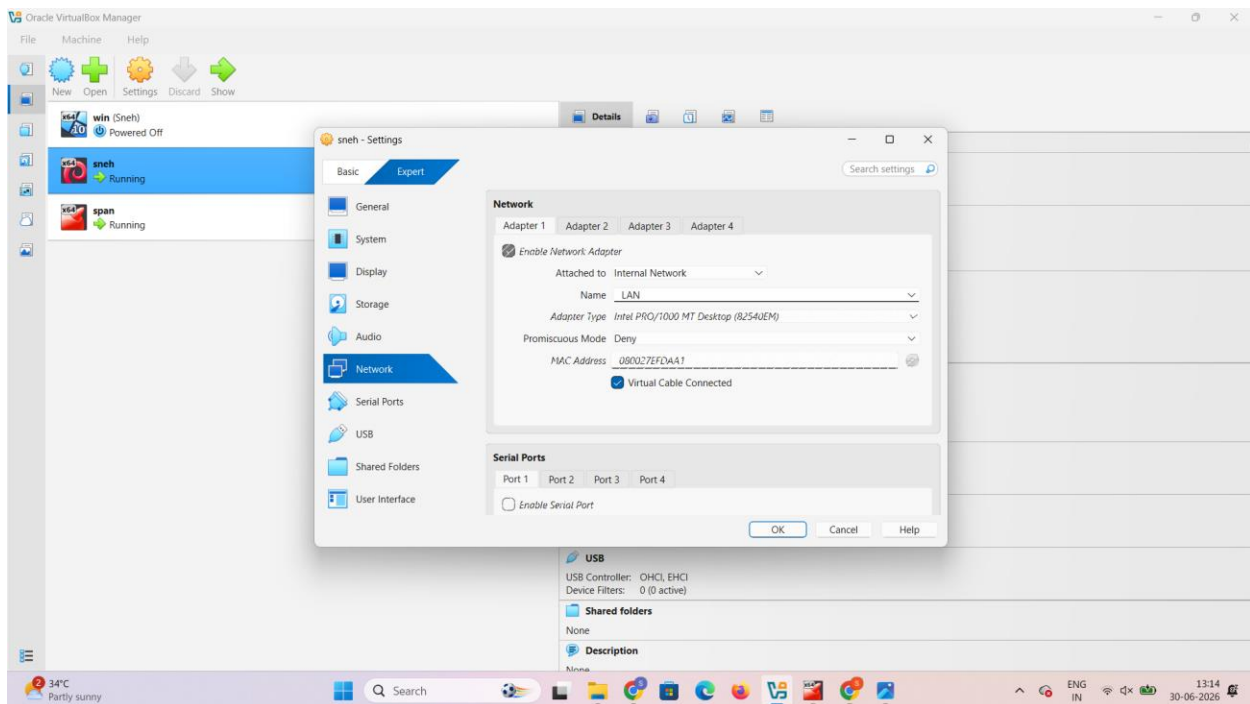


Figure 3.2 – sneh (Kali) Adapter 1 attached to Internal Network “LAN”.

- **Kali Linux (sneh) – Adapter 2:** Attached to NAT, providing the client itself with general internet access for updates/tools, separate from the lab LAN segment.

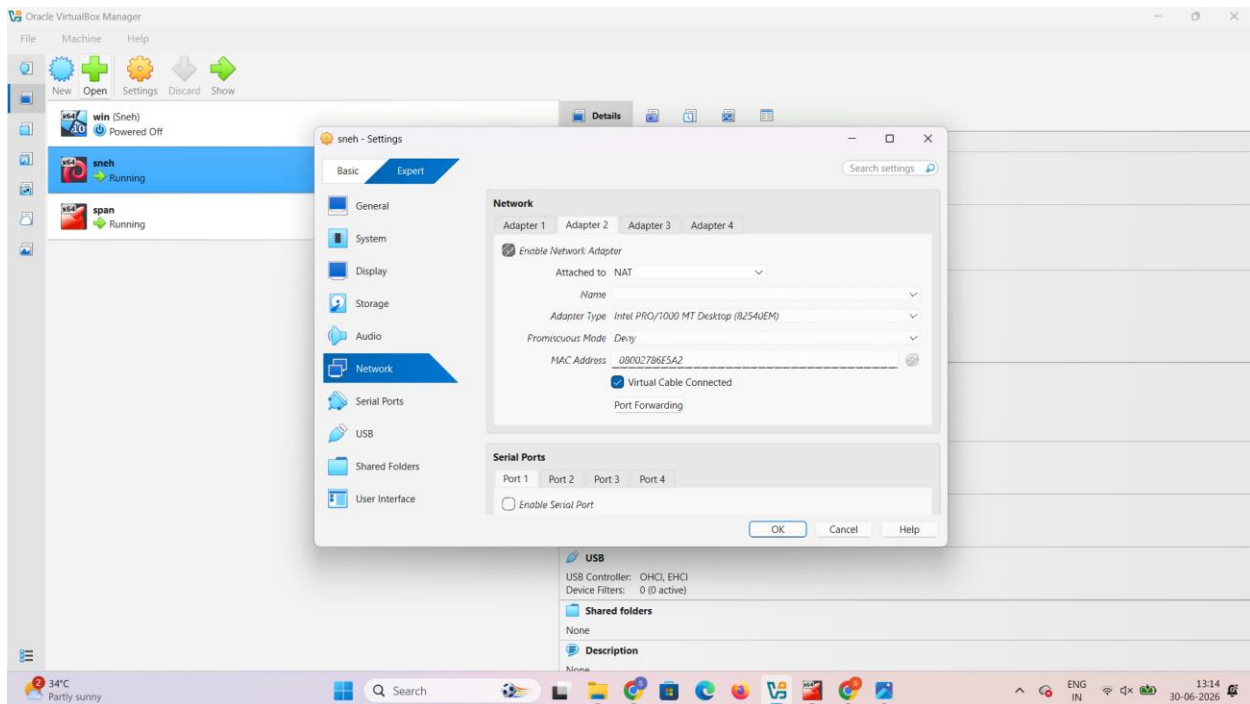


Figure 3.3 – sneh (Kali) Adapter 2 attached to NAT.

Step 4 – Start the VM (5 Minutes)

The OPNsense VM was powered on and allowed to boot from the attached ISO into the installer / live console.

Step 5 – Assign Interfaces (15 Minutes)

During first boot, OPNsense prompted for interface assignment. The first network adapter (NAT) was assigned as WAN (em1) and the second adapter (Internal Network) was assigned as LAN (em0). The changes were accepted, after which the console confirmed both interfaces were active.

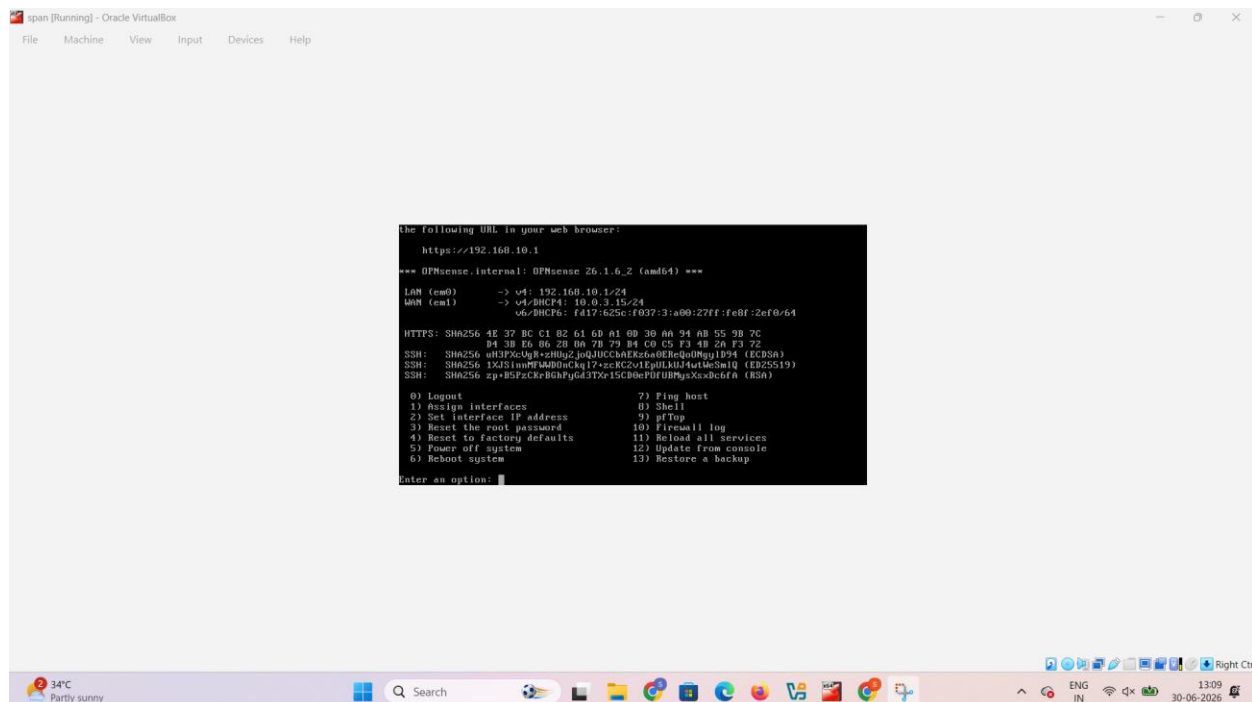


Figure 5.1 – OPNsense console confirming interface assignment: LAN (em0) and WAN (em1) (DHCP), with the web GUI URL displayed.

Step 6 – Configure the LAN Interface (15 Minutes)

The LAN interface (em0) was configured with a static IP address of 192.168.10.1/24. The DHCP server was enabled on the LAN interface with a range from 192.168.10.100 to 192.168.10.200, so that client VMs on the internal network automatically receive an IP address.

Console confirmation after configuration:

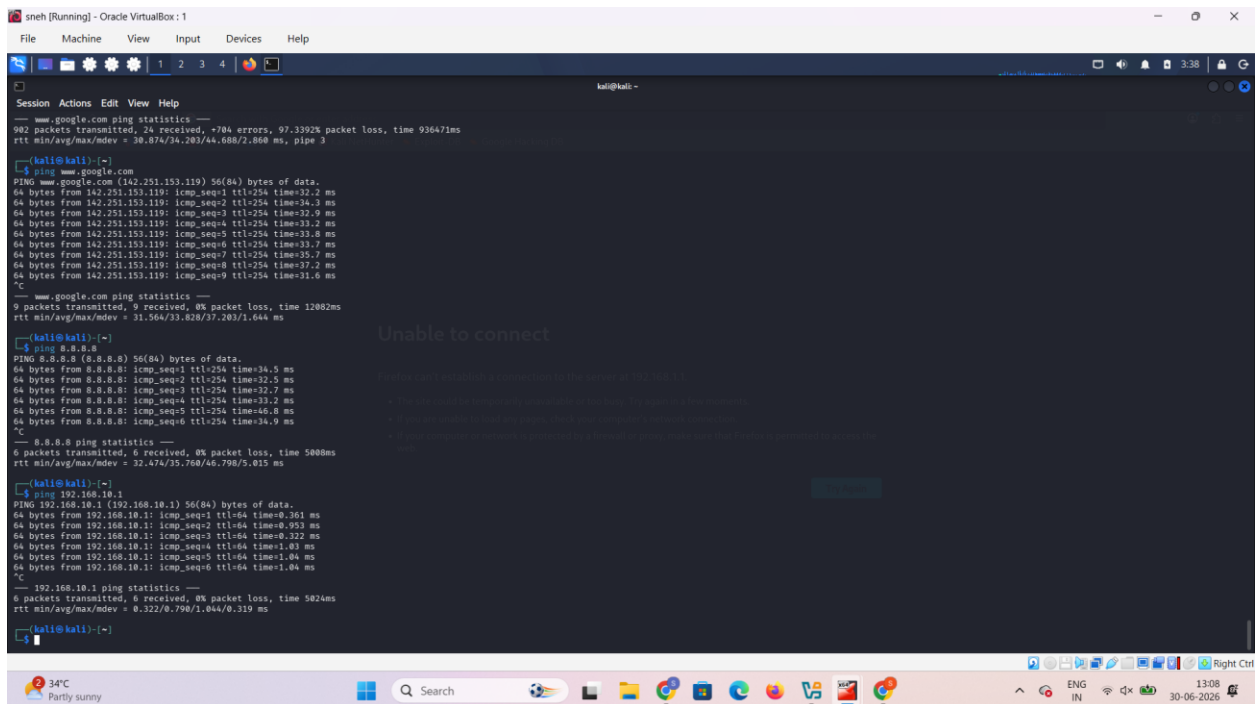
- **LAN (em0):** v4: 192.168.10.1/24
- **WAN (em1):** v4/DHCP4 address obtained automatically from the NAT network

Step 7 – Create the Client VM (15 Minutes)

The Kali Linux client VM (“sneh”) was configured with its primary network adapter attached to the same Internal Network (“LAN”) as OPNsense's LAN interface, then started. Once booted, the client was confirmed to receive an IP address automatically from the OPNsense DHCP server on the 192.168.10.0/24 range.

Step 8 – Verify Connectivity (10 Minutes)

From the Kali Linux client, connectivity to the firewall's LAN interface was verified using ping. The output below confirms successful, low-latency replies from 192.168.10.1 with 0% packet loss, proving that the client is correctly connected to the LAN segment and receiving traffic from the firewall.



```
sneh [Running] - Oracle VM VirtualBox : 1
File Machine View Input Devices Help

kali@kali: ~
Session Actions Edit View Help

--- www.google.com ping statistics ---
902 packets transmitted, 24 received, -70% errors, 97.3302% packet loss, time 93647ms
rtt min/avg/max/mdev = 10.074/94.203/44.688/2.868 ms, pipe 3

kali@kali: ~
$ ping www.google.com
PING www.google.com (142.251.153.119) 56(84) bytes of data:
64 bytes from 142.251.153.119: icmp_seq=1 ttl=254 time=32.2 ms
64 bytes from 142.251.153.119: icmp_seq=2 ttl=254 time=34.3 ms
64 bytes from 142.251.153.119: icmp_seq=3 ttl=254 time=32.9 ms
64 bytes from 142.251.153.119: icmp_seq=4 ttl=254 time=33.2 ms
64 bytes from 142.251.153.119: icmp_seq=5 ttl=254 time=33.8 ms
64 bytes from 142.251.153.119: icmp_seq=6 ttl=254 time=33.7 ms
64 bytes from 142.251.153.119: icmp_seq=7 ttl=254 time=35.7 ms
64 bytes from 142.251.153.119: icmp_seq=8 ttl=254 time=37.2 ms
64 bytes from 142.251.153.119: icmp_seq=9 ttl=254 time=31.6 ms
^C
--- www.google.com ping statistics ---
9 packets transmitted, 9 received, 0% packet loss, time 12082ms
rtt min/avg/max/mdev = 31.564/33.820/37.203/1.644 ms

kali@kali: ~
$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=254 time=34.5 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=254 time=32.5 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=254 time=32.7 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=254 time=33.2 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=254 time=46.8 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=254 time=34.9 ms
^C
--- 8.8.8.8 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5000ms
rtt min/avg/max/mdev = 32.474/35.760/46.790/5.015 ms

kali@kali: ~
$ ping 192.168.10.1
PING 192.168.10.1 (192.168.10.1) 56(84) bytes of data:
64 bytes from 192.168.10.1: icmp_seq=1 ttl=64 time=0.361 ms
64 bytes from 192.168.10.1: icmp_seq=2 ttl=64 time=0.953 ms
64 bytes from 192.168.10.1: icmp_seq=3 ttl=64 time=0.322 ms
64 bytes from 192.168.10.1: icmp_seq=4 ttl=64 time=1.03 ms
64 bytes from 192.168.10.1: icmp_seq=5 ttl=64 time=1.04 ms
64 bytes from 192.168.10.1: icmp_seq=6 ttl=64 time=1.04 ms
^C
--- 192.168.10.1 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5024ms
rtt min/avg/max/mdev = 0.322/0.790/1.044/0.319 ms

kali@kali: ~
```

Figure 8.1 – Successful ping to the firewall (192.168.10.1): 0% packet loss, confirming LAN connectivity.

Step 9 – Access the OPNsense Web Interface (10 Minutes)

A browser on the Kali client was pointed to <https://192.168.10.1>. The self-signed certificate warning was accepted, and login was completed using the default credentials (root / opnsense), giving access to the OPNsense web GUI.

Step 10 – Run the Setup Wizard (20 Minutes)

The initial setup wizard was completed, configuring the hostname, domain name, DNS settings, time zone, and a new administrator password, after which the dashboard became available.

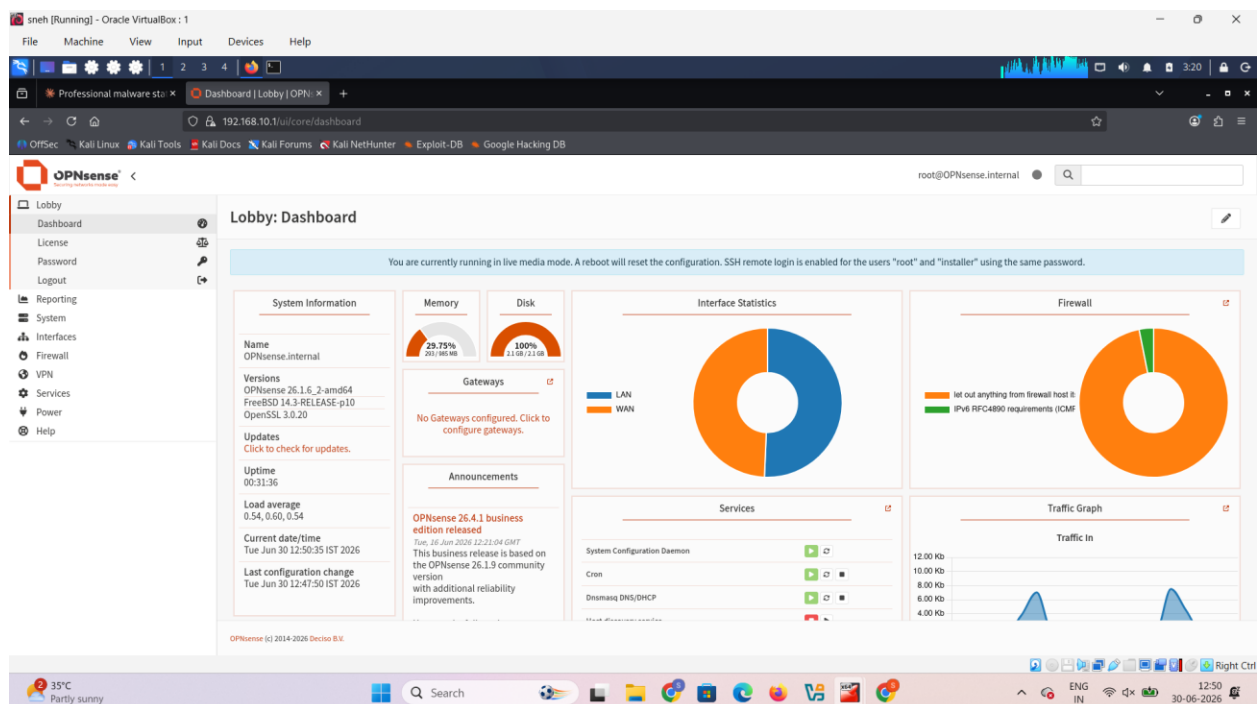
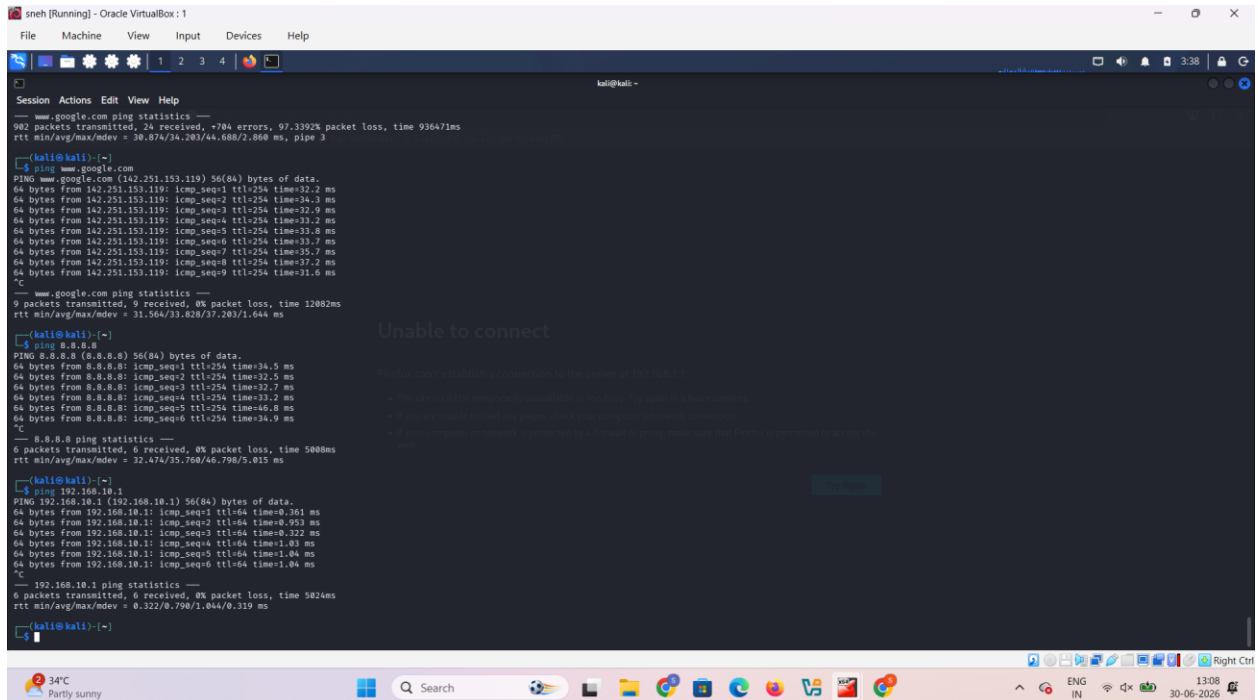


Figure 10.1 – OPNsense web GUI Dashboard after successful login and setup, showing system information, interface statistics (LAN/WAN), and active services.

Step 11 – Verify Internet Access (10 Minutes)

From the Kali Linux client, internet reachability through the firewall's WAN interface was tested by pinging a public IP address (8.8.8.8) and a domain name (google.com / www.google.com). Both tests succeeded with 0% packet loss, confirming that NAT and routing through OPNsense are functioning correctly and that DNS resolution works for the client.



```
sneh [Running] - Oracle VirtualBox : 1
File Machine View Input Devices Help

kali@kali: ~
-- www.google.com ping statistics --
982 packets transmitted, 24 received, 97.7592% packet loss, time 93647ms
rtt min/avg/max/mdev = 38.874/34.203/44.688/2.860 ms, pipe 3

kali@kali: ~
ping www.google.com
PING www.google.com (142.251.153.119) 56(84) bytes of data:
64 bytes from 142.251.153.119: icmp_seq=1 ttl=254 time=32.2 ms
64 bytes from 142.251.153.119: icmp_seq=2 ttl=254 time=34.3 ms
64 bytes from 142.251.153.119: icmp_seq=3 ttl=254 time=32.9 ms
64 bytes from 142.251.153.119: icmp_seq=4 ttl=254 time=33.2 ms
64 bytes from 142.251.153.119: icmp_seq=5 ttl=254 time=33.8 ms
64 bytes from 142.251.153.119: icmp_seq=6 ttl=254 time=33.7 ms
64 bytes from 142.251.153.119: icmp_seq=7 ttl=254 time=35.7 ms
64 bytes from 142.251.153.119: icmp_seq=8 ttl=254 time=37.2 ms
64 bytes from 142.251.153.119: icmp_seq=9 ttl=254 time=31.6 ms
^C
-- www.google.com ping statistics --
9 packets transmitted, 9 received, 0% packet loss, time 12082ms
rtt min/avg/max/mdev = 31.564/33.828/37.203/1.644 ms

kali@kali: ~
ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=254 time=34.5 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=254 time=32.5 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=254 time=32.7 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=254 time=33.2 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=254 time=46.8 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=254 time=34.9 ms
^C
-- 8.8.8.8 ping statistics --
6 packets transmitted, 6 received, 0% packet loss, time 5080ms
rtt min/avg/max/mdev = 32.474/35.768/46.798/5.015 ms

kali@kali: ~
ping 192.168.10.1
PING 192.168.10.1 (192.168.10.1) 56(84) bytes of data:
64 bytes from 192.168.10.1: icmp_seq=1 ttl=64 time=0.361 ms
64 bytes from 192.168.10.1: icmp_seq=2 ttl=64 time=0.953 ms
64 bytes from 192.168.10.1: icmp_seq=3 ttl=64 time=0.222 ms
64 bytes from 192.168.10.1: icmp_seq=4 ttl=64 time=1.03 ms
64 bytes from 192.168.10.1: icmp_seq=5 ttl=64 time=1.04 ms
64 bytes from 192.168.10.1: icmp_seq=6 ttl=64 time=1.04 ms
^C
-- 192.168.10.1 ping statistics --
6 packets transmitted, 6 received, 0% packet loss, time 5824ms
rtt min/avg/max/mdev = 0.322/0.798/1.044/0.319 ms

kali@kali: ~
```

Figure 11.1 – Successful ping to 8.8.8.8 and www.google.com from the Kali client, confirming working internet access through OPNsense.

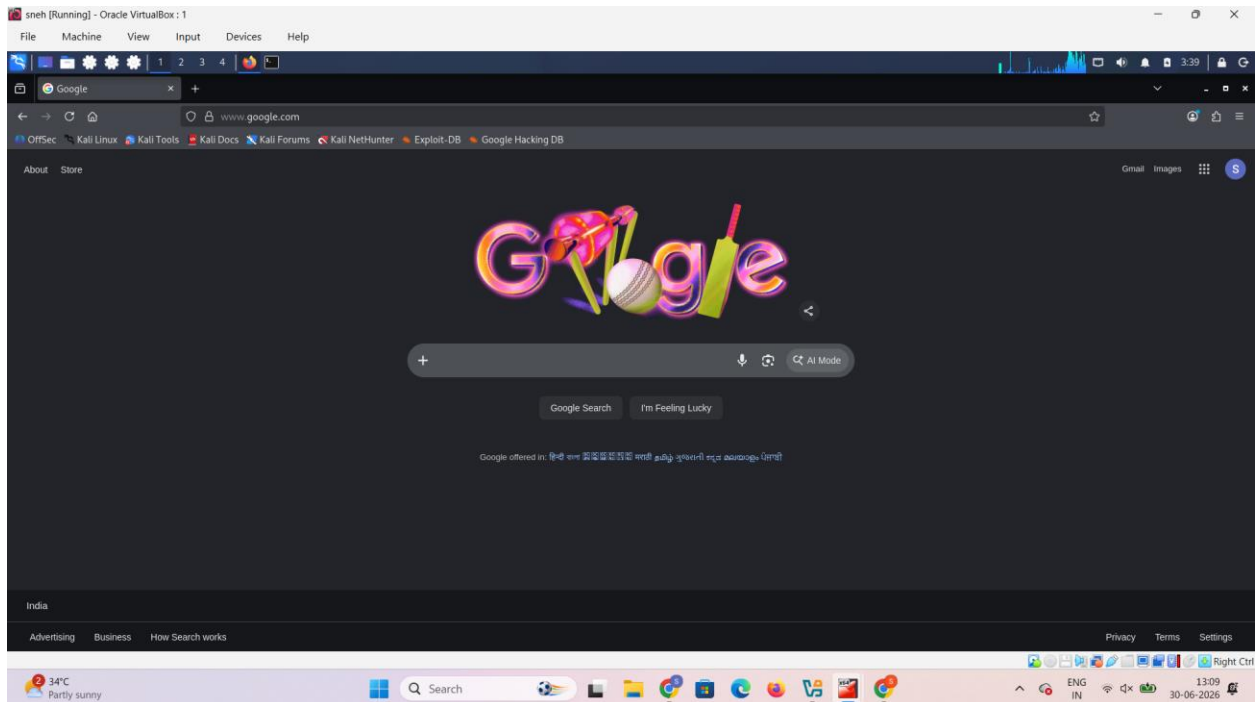


Figure 11.2 – Successful browsing to www.google.com from the Kali client, confirming end-to-end internet access via the firewall.

Step 12 – Explore the Dashboard *(15 Minutes)*

The OPNsense web GUI was explored section by section. A one-sentence summary of the purpose of each section is given below:

- **Dashboard:** Provides an at-a-glance overview of system status, including CPU/memory/disk usage, interface traffic, and active services.
- **Interfaces:** Used to view and configure the network interfaces (e.g., WAN, LAN) including IP addressing and interface-specific settings.
- **Firewall:** Used to create, edit, and manage firewall rules that control which traffic is allowed or blocked between interfaces.
- **Aliases:** Allows grouping of IP addresses, networks, or ports under a single reusable name to simplify firewall rule management.
- **NAT:** Used to configure Network Address Translation rules, such as port forwarding and outbound NAT, that translate addresses between the LAN and WAN.
- **Services:** Manages built-in services such as the DHCP server, DNS resolver/forwarder, and other system daemons running on the firewall.
- **VPN:** Used to configure virtual private network technologies (e.g., OpenVPN, IPsec, WireGuard) for secure remote or site-to-site connectivity.
- **Reporting:** Displays historical traffic and usage statistics, helping administrators analyze network activity over time.
- **System:** Contains general system settings such as firmware updates, user/group management, certificates, and general configuration.
- **Firewall Logs:** Displays a real-time and historical log of traffic that has been evaluated against firewall rules, useful for troubleshooting and auditing.

One-Page Report

1. What is OPNsense?

OPNsense is a free and open-source firewall and routing platform based on FreeBSD. It provides a web-based graphical interface for managing firewall rules, network address translation (NAT), virtual private networks (VPN), traffic shaping, and intrusion detection. Built as a fork of pfSense/m0n0wall, OPNsense is widely used in both home labs and enterprise environments because it combines enterprise-grade features with an easy-to-use interface, regular security updates, and strong community support.

2. Why Are Two Network Interfaces Required?

A firewall's core purpose is to control and inspect traffic moving between two or more separate networks of differing trust levels. To do this, it must have at least one interface connected to each network it is meant to separate. In this lab, OPNsense uses two interfaces: one connected to the external/untrusted network (WAN, representing the internet) and one connected to the internal/trusted network (LAN, where client machines such as Windows and Kali Linux reside). Without two distinct interfaces, the firewall would have no way to distinguish between traffic originating from trusted internal hosts and traffic arriving from the untrusted internet, making it impossible to enforce security policies between them.

3. Difference Between the WAN and LAN Interfaces

- **WAN (Wide Area Network):** Connects the firewall to the external network (the internet, simulated here using VirtualBox's NAT adapter). It typically obtains its IP address dynamically via DHCP from the upstream network and is treated as untrusted, meaning incoming traffic from it is filtered and restricted by default.
- **LAN (Local Area Network):** Connects the firewall to the internal/private network (the VirtualBox Internal Network named "LAN," shared with the Windows and Kali client VMs). It is assigned a static IP address (192.168.10.1/24) and is treated as trusted, allowing internal clients to initiate outbound connections freely while incoming connections from the WAN side are blocked unless explicitly permitted.

4. Why Is a Firewall Placed Between the Internet and the Internal Network?

Placing a firewall between the internet and the internal network creates a controlled checkpoint through which all traffic must pass before reaching protected systems. This allows the firewall to inspect, filter, and log traffic, blocking malicious or unauthorized connection attempts from the internet while still allowing legitimate outbound access for internal users. It also enables Network Address Translation (NAT), which lets multiple internal devices share a single public IP address, hides the internal network structure from external attackers, and allows administrators to enforce security policies, such as restricting which ports or services are reachable from outside. In short, the firewall acts as the first line of defense, reducing the internal network's exposure to threats originating from the public internet.